

Lab 2 — Code Hint Sheet

A 1D CNN on Real Clinical Spectra

Stuck on a **YOUR TURN** cell? Each blank you need is below with a few options. Pick one, type it in, and **run the cell** — the built-in `assert` checks will tell you whether it is right. Only the five marked cells need editing; run everything else as-is.

Step 3 — the training loop. Write these five lines *in order* inside the batch loop (Lecture 2: forward → loss → zero → backward → step) — same recipe as Lab 1, reused here for spectra.

Blank 1 — the five-line loop body.

- (a) `pred = model(X_batch)` then `loss = loss_fn(pred, y_batch)` then `optimizer.zero_grad()` then `loss.backward()` then `optimizer.step()`
 - (b) `pred = loss_fn(X_batch)` then `loss = model(pred, y_batch)` ...
 - (c) `loss.backward()` then `pred = model(X_batch)` then `optimizer.step()` ...
- (a) is the Lecture 2 order; (b) swaps what `model` and `loss_fn` do; (c) backs up before making a guess — there is nothing to backprop yet.

Step 4 — the first conv block. One input channel (a raw spectrum), 8 filters, a 15-bin-wide filter, and padding so the length is unchanged (Lecture 5's code slide).

Blank 2 — `self.conv1`.

- (a) `nn.Conv1d(in_channels=1, out_channels=8, kernel_size=15, padding=7)`
- (b) `nn.Conv1d(in_channels=8, out_channels=1, kernel_size=15, padding=7)`
- (c) `nn.Linear(in_features=1, out_features=8)`

Step 4 — the second conv block. Mirror block 1's pattern, but now reading the 8 channels block 1 produced and turning them into 16.

Blank 3 — `self.conv2`, `self.relu2`, `self.pool2`.

- (a) `nn.Conv1d(in_channels=8, out_channels=16, kernel_size=15, padding=7)`
then, same as block 1: `nn.ReLU()` and `nn.MaxPool1d(4)`
- (b) `nn.Conv1d(in_channels=16, out_channels=8, kernel_size=15, padding=7)`
- (c) `nn.Conv1d(in_channels=8, out_channels=16, kernel_size=1, padding=0)`

Step 4 — the input size of the first Linear layer. The `Conv1d` layers keep the length unchanged (that's the padding); each `MaxPool1d(4)` divides the length by 4, and there are **two** of them; `Flatten` then stacks all 16 channels into one vector. So: $16 \times (6000 \div 4 \div 4)$.

Blank 4 — `self.fc1`.

- (a) `self.fc1 = nn.Linear(16 * (6000 // 4 // 4), 32)`
 - (b) `self.fc1 = nn.Linear(16 * 6000, 32)`
 - (c) `self.fc1 = nn.Linear(8 * 375, 32)`
- (a) is right: two `MaxPool1d(4)`s take $6000 \rightarrow 1500 \rightarrow 375$, and 16 channels give $16 \times 375 = 6000$ features. (b) forgets the pooling shrank the length; (c) uses the wrong channel count (8, not 16) — both make `fc1` the wrong size and the forward pass will error.

Step 5 — pick a batch size. A middle-ground value for 590 training spectra (Lecture 4's Goldilocks range).

Blank 5 — `CNN_BATCH_SIZE`.

- (a) `CNN_BATCH_SIZE = 32`
 - (b) `CNN_BATCH_SIZE = 1`
 - (c) `CNN_BATCH_SIZE = 590`
- (a) and other values in roughly 8–64 all pass the check and train fine — (b) updates extremely often but painfully slowly, (c) is the whole training set at once, no mini-batching at all.

Step 8 — commit to a threshold. From your Step 7 exploration, pick a cut that catches at least *half* the resistant cases (recall ≥ 0.5). On this trained model recall stays 0 near 0.5 and climbs only for low cuts.

Blank 6 — `chosen_threshold`.

- (a) `chosen_threshold = 0.05`
 - (b) `chosen_threshold = 0.5`
 - (c) `chosen_threshold = 0.9`
- (a) reaches recall ≈ 0.67 — it clears the `assert recall_at_cut >= 0.5`. (b) is the default where recall ≈ 0 ; (c) is even stricter, still 0. Any low cut that hits recall ≥ 0.5 passes — there is no single "right" number, it's the clinical trade-off.

*Answers self-check in the notebook: the shape cell confirms `fc1` accepts the right number of features and the forward pass returns one logit per spectrum; the batch-size and threshold cells print their own success messages. Watch the **resistant recall** number, not just accuracy — a model that always says "susceptible" can still score $\sim 94\%$ accuracy while catching zero real cases.*